Vulnerability Detection - Best Effort

Aravind Machiry

We acknowledge funding provided by Microelectronic Commons and the support of OUSW(R&E).

Bits and pieces copied from: https://help.semmle.com/QL/ql-training/cpp

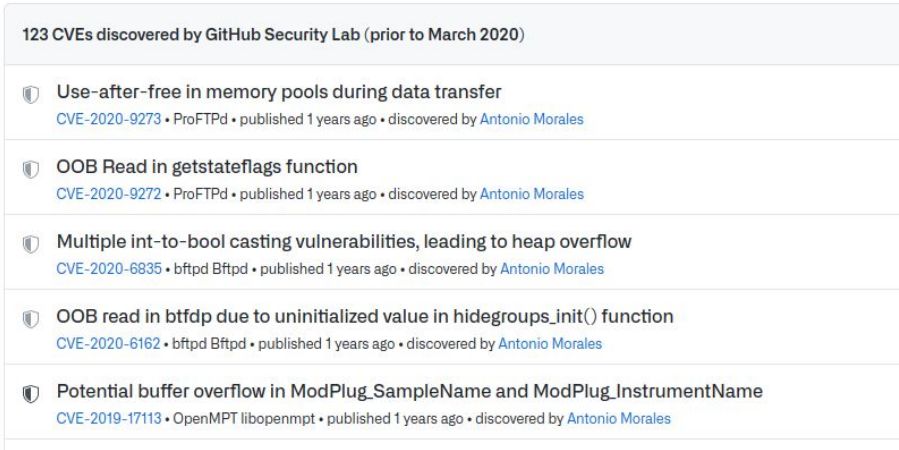
Best effort techniques

- We don’t need to find all the bugs, i.e., we do not need to be sound.
    - “Unsoundness was controversial in the research community, though it has since become almost a de facto tool bias for commercial products and many research projects.”
- Precision (i.e., no false positives is more important):
    - “False positives do matter. In our experience, more than 30% easily cause problems. People ignore the tool.”

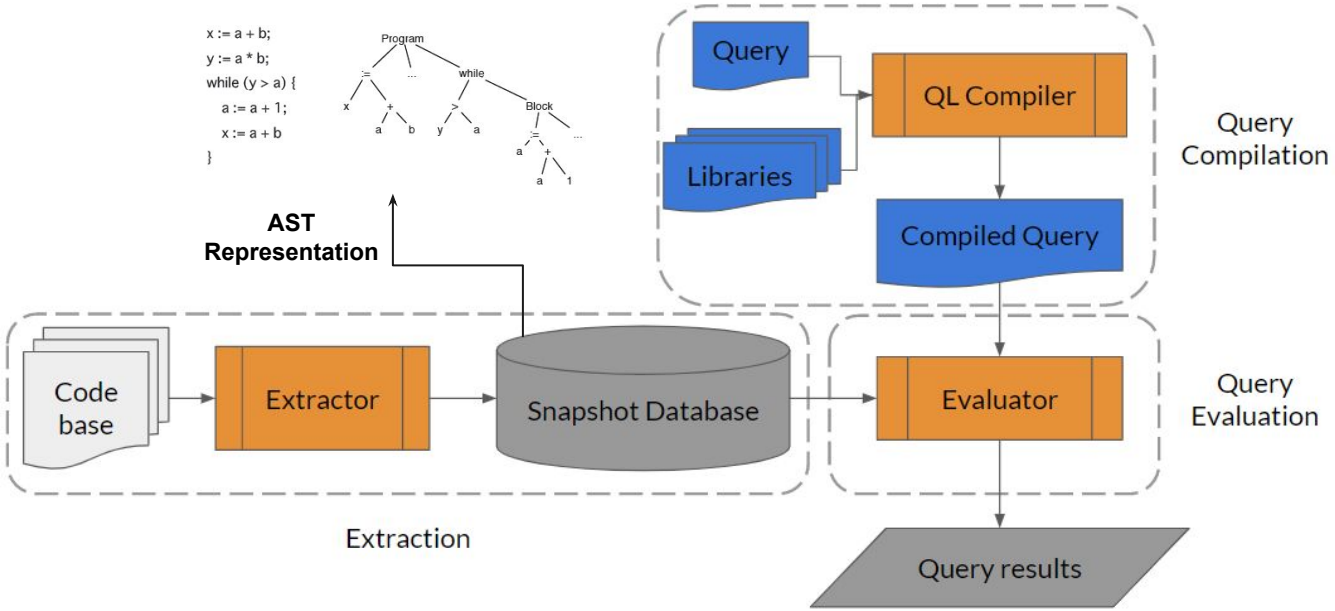
*a-few-billion-lines-of-code-later.

CodeQL: State of the art (Pattern based)

- Logical query language similar to SQL.
- Modular and extensible:
  - Support for custom classes.
  - Libraries.
- Well maintained by Microsoft and its open source.



CodeQL


x := a + b;
y := a * b;
while (y > a) {
a := a + 1;
x := a + b
}
Program
:=
...
while
x
+
>
Block
a
b
y
a
:=
...
a
+
a
1
AST
Representation
Query
QL Compiler
Libraries
Compiled Query
Query
Compilation
Code
base
Extractor
Snapshot Database
Evaluator
Query
Evaluation
Extraction
Query results

CodeQL : Example

```
void fire_thrusters(double vectors[12]) {

  for (int i = 0; i < 12; i++) {

    ... vectors[i] ...

  }

}

...

double thruster[3] = ... ;

fire_thrusters(thruster);
```

- What is the problem with the code?

CodeQL : Example

```
void fire_thrusters(double vectors[12]) {

  for (int i = 0; i < 12; i++) {

    ... vectors[i] ...

  }

}

...

double thruster[3] = ... ;

fire_thrusters(thruster);
```

- What is the problem with the code?
- In C, array types of parameters degrade to pointer types.
- The size is ignored!

Finding using CodeQL

```
void fire_thrusters(double vectors[12]) {

  for (int i = 0; i < 12; i++) {

    ... vectors[i] ...

  }

}

...

double thruster[3] = ... ;

fire_thrusters(thruster);
```

- Basic Query Structure:

import <language library>

from <entities and types>

where <queries on entities>

select <entities to select>

Finding using CodeQL

```
void fire_thrusters(double vectors[12]) {
  for (int i = 0; i < 12; i++) {
    ... vectors[i] ...
  }
}
...
double thruster[3] = ... ;
fire_thrusters(thruster);
```

- First, let's find function calls.

```
import cpp
from Function f, FunctionCall c
where f = c.getTarget()
select "Found call to " + f.getName()
```

Finding using CodeQL

```
void fire_thrusters(double vectors[12]) {

  for (int i = 0; i < 12; i++) {

    ... vectors[i] ...

  }

}

...

double thruster[3] = ... ;

fire_thrusters(thruster);
```

- Argument should be constant array.

```
import cpp

from Function f, FunctionCall c, int i

where f = c.getTarget()

 and (f.getParameter(i).getType() instanceof ArrayType)

select "Found call to " + f.getName()
```

Finding using CodeQL

```
void fire_thrusters(double vectors[12]) {
  for (int i = 0; i < 12; i++) {
    ... vectors[i] ...
  }
}
...
double thruster[3] = ... ;
fire_thrusters(thruster);
```

Checking size of parameter and argument:

```
import cpp
from Function f, FunctionCall c, int i, int a, int b
where f = c.getTarget()
 and a = c.getArgument(i).getType().(ArrayType).getArraySize()
 and b = f.getParameter(i).getType().(ArrayType).getArraySize()
 and a < b
select c.getArgument(i), f, “Found vulnerability”
```

Finding using CodeQL

```
void fire_thrusters(double vectors[12]) {
  for (int i = 0; i < 12; i++) {
    ... vectors[i] ...
  }
}
...
double thruster[3] = ... ;
fire_thrusters(thruster);
```

Checking size of parameter and argument:

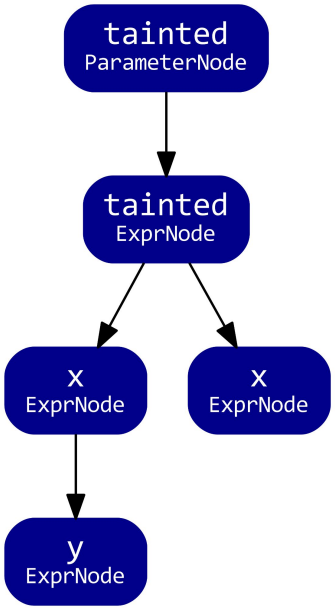
Characteristic predicate

Argument size < parameter size

```
import cpp
from Function f, FunctionCall c, int i, int a, int b
where f = c.getTarget()
 and a = c.getArgument(i).getType().(ArrayType).getArraySize()
 and b = f.getParameter(i).getType().(ArrayType).getArraySize()
 and a < b
select c.getArgument(i), f, “Found vulnerability”
```

CodeQL : Data flow analysis

```
int func(int tainted) {
   int x = tainted;
   if (someCondition) {
     int y = x;
     callFoo(y);
   } else {
     return x;
   }
  return -1;
}
```



CodeQL : Data flow example

Format string checker: Use of externally controlled format string.

```
printf(userControlledString, arg1);
```

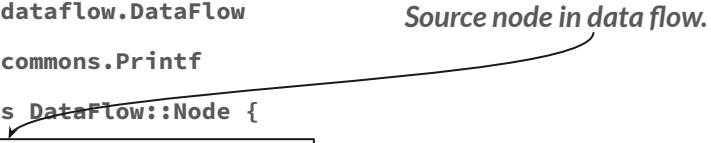
Goal: Find uses of printf (or similar) where the format string can be controlled by an attacker.

CodeQL : format string checker

```
import cpp
import semmle.code.cpp.dataflow.DataFlow
import semmle.code.cpp.commons.Printf
class SourceNode extends DataFlow::Node {
    SourceNode() {
      not DataFlow::localFlowStep(_, this)
    }
}
from FormattingFunction f, Call c, SourceNode src, DataFlow::Node  arg
where c.getTarget() = f and arg.asExpr() = c.getArgument(f.getFormatParameterIndex()) and
      DataFlow::localFlow(src, arg) and not src.asExpr() instanceof StringLiteral
select arg, "Non-constant format string."
```

CodeQL : format string checker

```
import cpp
import semmle.code.cpp.dataflow.DataFlow
import semmle.code.cpp.commons.Printf
class SourceNode extends DataFlow::Node {
    SourceNode() {
      not DataFlow::localFlowStep(_, this)
    }
}
from FormattingFunction f, Call c, SourceNode src, DataFlow::Node  arg
where c.getTarget() = f and arg.asExpr() = c.getArgument(f.getFormatParameterIndex()) and
      DataFlow::localFlow(src, arg) and not src.asExpr() instanceof StringLiteral
select arg, "Non-constant format string."
```



CodeQL : format string checker

```
import cpp
import semmle.code.cpp.dataflow.DataFlow
import semmle.code.cpp.commons.Printf
class SourceNode extends DataFlow::Node {
  SourceNode() {
    not DataFlow::localFlowStep(_, this)
  }
}
from FormattingFunction f, Call c, SourceNode src, DataFlow::Node  arg
where c.getTarget() = f and arg.asExpr() = c.getArgument(f.getFormatParameterIndex()) and
      DataFlow::localFlow(src, arg) and not src.asExpr() instanceof StringLiteral
select arg, "Non-constant format string."
```

Source node in data flow.

Flows from src to arg.

CodeQL : format string checker

```
import cpp
import semmle.code.cpp.dataflow.DataFlow
import semmle.code.cpp.commons.Printf
class SourceNode extends DataFlow::Node {
  SourceNode() {
    not DataFlow::localFlowStep(_, this)
  }
}
from FormattingFunction f, Call c, SourceNode src, DataFlow::Node  arg
where c.getTarget() = f and arg.asExpr() = c.getArgument(f.getFormatParameterIndex()) and
      DataFlow::localFlow(src, arg) and not src.asExpr() instanceof StringLiteral
select arg, "Non-constant format string."
```

Source node in data flow.

Flows from src to arg.

Src is not a constant string.

CodeQL: Other features

- Taint analysis:
  - Custom taint source functions and taint sinks.

- Also, supports various other languages: C++, Java, JavaScript, etc.

CodeQL: Drawbacks

- AST representation (v/s IR representation):
  - Depends on coding pattern.

- Not suitable for complex analysis involving fine-grained data flow.